

Certificats pour les preuves d'équivalence de circuits avec Isabelle/HOL

Alexis Beaucourt

juin-juillet 2026

Table des matières

1	Introduction	2
1.1	Abstract	2
1.2	Définition des PRO/PROP	2
1.3	Normalisation des PRO et des Perm	3
1.4	Contribution	4
2	Implémentation des certificats	5
2.1	Le simplificateur d'Isabelle	5
2.2	Equivalence des PRO	6
2.3	Equivalence des permutations	7
3	Tests de performance	8
3.1	Méthodologie	8
3.2	Résultats pour les PRO	9
3.3	Résultats pour les permutations	10
4	Extension aux PROP	11
4.1	Numérotation	11
4.2	Forme normale	12
4.3	Système de réécriture	13
5	Conclusion	15
6	Remerciments	15
7	Annexes	17
7.1	Exemple : profondeur des portes	20
7.2	Exemple : vraie profondeur	21
7.3	Exemple : priorité des fils	21
7.4	Exemple : forme normale	22

1 Introduction

1.1 Abstract

Les diagrammes sont fréquemment utilisés à des fins de raisonnement, notamment parce qu'ils apportent une représentation visuelle de l'objet étudié. Malheureusement, les diagrammes ne donnent souvent pas de représentation unique ; la question est alors de déterminer si deux diagrammes sont équivalents.

Comme les raisonnements sur les diagrammes sont souvent longs et composés de petites étapes simples, on souhaiterait un processus entièrement automatique, c'est-à-dire qui décide l'équivalence, et produit une preuve dans le cas positif.

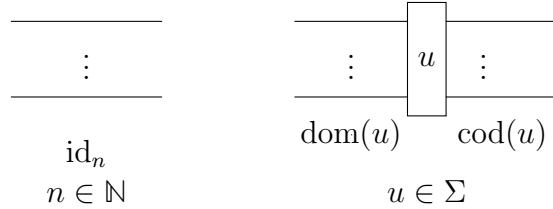
On s'intéressera à un certain type de diagramme, appelé circuit, qui est très utilisé dans la programmation quantique [3]. On le définit d'ailleurs dans la partie suivante.

La méthode ici choisie pour décider l'équivalence est celle de [2] et est explicitée en Section 1.3. Elle considère les circuits comme des termes et réduit ces derniers à une forme normale par un système de réécriture. Il existe cependant d'autres méthodes, qui consistent à voir les circuits comme des graphes [4] ou des hypergraphes [1]. L'avantage de cette méthode par rapport aux autres est qu'elle donne un certificat en cas d'équivalence, sous la forme d'une preuve formelle en Isabelle/HOL.

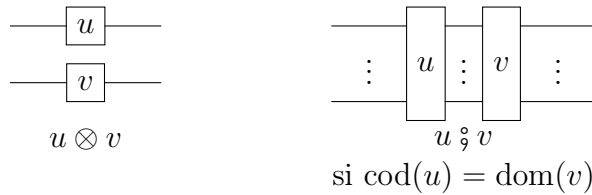
1.2 Définition des PRO/PROP

Chaque circuit u possède un domaine $\text{dom}(u) \in \mathbb{N}$ et un codomaine $\text{cod}(u) \in \mathbb{N}$

On définit les circuits de $\mathbf{PRO}_{\Sigma, \mathcal{E}}$ comme étant les circuits formés à partir des générateurs suivants :

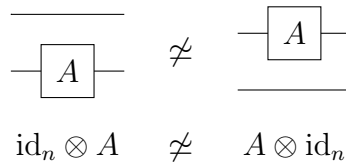


Ainsi que les opérations suivantes :

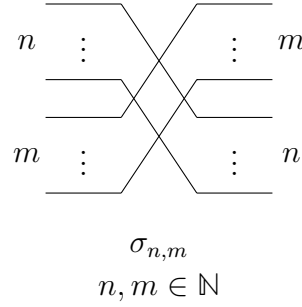


Cependant, on quotiente le tout par les égalités de $\mathcal{E}$, en plus des égalités (1) à (5) de la [Figure 1](#)

Il est important de noter que l'ordre des fils d'entrée et de sortie est important, dans le sens où il n'est pas permis de les permuter. En particulier :



On peut ensuite définir les $\mathbf{PROP}_{\Sigma, \mathcal{E}}$ comme étant des $\mathbf{PRO}_{\Sigma \cup \Sigma_0, \mathcal{E} \cup \mathcal{E}_0}$, où Σ_0 contient les générateurs suivants (appelés swaps) :



Et les $\mathcal{E}_0$ sont les égalités (6) à (10) de la [Figure 1](#)

On définit également les permutations, qui sont les $\mathbf{PROP}_{\emptyset, \emptyset}$, c'est-à-dire les circuits uniquement composés de swaps. Un tel circuit peut donc être réduit à la permutation qu'il opère sur ses entrées.

Pour le restant de ce rapport, on notera $\simeq$ la relation d'équivalence entre deux termes.

1.3 Normalisation des PRO et des Perm

Afin de décider l'équivalence $t_1 \simeq t_2$, il convient de définir une fonction de normalisation sur les circuits. Cette fonction est de plus exprimée sous forme de système de réécriture, dont on peut prouver qu'il est terminant et confluent.

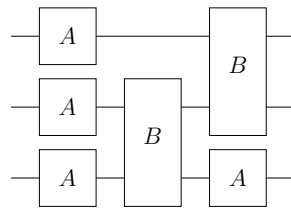
Pour simplifier les notations, on pose $L_\ell^k(g) = \text{id}_k \otimes g \otimes \text{id}_\ell$

Ainsi que le défaut $\text{def}(u) = \text{dom}(u) - \text{cod}(u)$

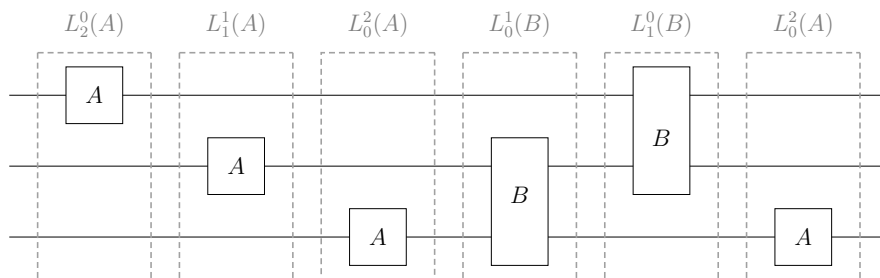
Théorème 1.3.1. *Le système défini en [Figure 2](#) est terminant, confluent et met les circuits de $\mathbf{PRO}_\Sigma$ en forme normale.*

Démonstration. Voir [2] □

Exemple 1.3.2. *Si on se munit du circuit suivant :*



Alors on obtient sa forme normale après application des règles :



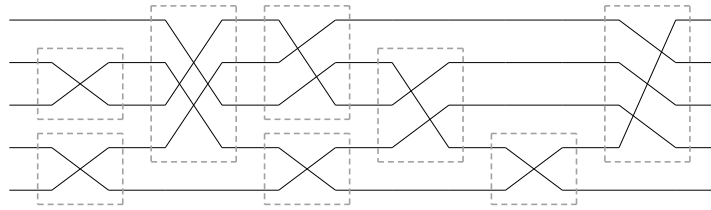
On remarque en particulier que le circuit en forme normale est découpé en couches contenant chacune une seule porte, et que ces couches sont arrangées de sorte que les portes les plus hautes soient le plus à gauche possible.

On définit un sorte spéciale de swap $\tau_n = \sigma_{n,1}$

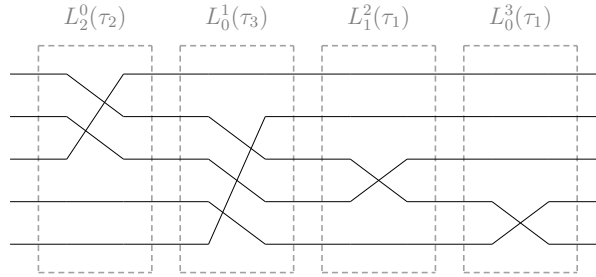
Théorème 1.3.3. *Le système composé des règles (R1) jusqu'à (R9) de la [Figure 2](#), ainsi que de celles de la [Figure 3](#) est terminant, confluent et met les circuits de $\mathbf{PROP}_\emptyset$ en forme normale.*

Démonstration. Voir [2] □

Exemple 1.3.4. *Si on se munit du circuit suivant :*



Alors on obtient sa forme normale après application des règles :



On remarque en particulier que le circuit en forme normale est découpé en couches (comme pour les $\mathbf{PRO}$) contenant chacune un seul τ_n , et que ces couches sont arrangées de sorte que le fil le plus haut modifié par la couche soit strictement décroissant au long du circuit.

1.4 Contribution

Durant mon stage, j'ai implémenté l'algorithme produisant les certificats d'équivalence pour les deux cas déjà traités en théorie par [2] (les $\mathbf{PRO}_\Sigma$ et les permutations $\mathbf{PROP}_\emptyset$), et j'ai fait des tests de performance pour avoir une idée de leur efficacité.

J'ai aussi trouvé un système de réécriture qui pourrait étendre les deux cas précédant aux $\mathbf{PROP}_\Sigma$, mais je n'ai pas eu le temps durant mon stage de formaliser ce dernier en Isabelle/HOL.

2 Implémentation des certificats

2.1 Le simplificateur d'Isabelle

En Isabelle/HOL la tactique `simp` permet de modifier (ou prouver) un but en appliquant des règles de réécriture. En particulier, `simp` peut être invoquée sous plusieurs formes :

- `simp` utilise un ensemble de règles de réécriture par défaut. Cet ensemble est un mix de règles judicieusement choisies par les créateurs de bibliothèques et de règles automatiquement générées par des mots clés comme `function`, `fun`, `inductive`, etc.
- `(simp add: regle1 regle2)` permet à l'utilisateur de rajouter ses propres règles de réécriture dans l'ensemble utilisé par `simp`
- `(simp only: regle1 regle2)` permet à l'utilisateur de ne pas utiliser les règles par défaut, mais seulement celles qu'il choisit
- `(simp cong: regle1 regle2)` permet à l'utilisateur de rajouter des règles de congruence

Les options `add`, `only` et `cong` peuvent être utilisées dans la même invocation de `simp`, par exemple `(simp only: regle1 cong: regle2)` n'utilise que 2 règles, dont une de congruence.

Une règle de réécriture est un lemme (qui doit être démontré dans Isabelle) sous la forme $P_1 \implies \dots \implies P_n \implies A = B$. Il représente la règle $A \rightarrow B$ si $P_1, \dots, P_n$

En pratique, le simplificateur va essayer de prouver les conditions $P_1, \dots, P_n$ en s'appuyant récursivement sur chacune des conditions, et réécrire A en B une fois qu'il a réussi. S'il n'arrive pas à prouver une des conditions (parce qu'il n'a pas assez de règles ou bien parce que la condition est fausse), alors il abandonne l'utilisation de cette règle et en essaye d'autres.

Une règle de congruence est un lemme de la forme $P_1 \implies \dots \implies P_n \implies f(x) = f(y)$. Elle s'applique avant les règles de réécriture, dès que le simplificateur reconnaît que les deux côtés de l'égalité commencent par f . De plus, les règles de congruence se distinguent des règles de réécriture par le fait qu'il ne s'agit pas ici de remplacer $f(x)$ par $f(y)$, mais de prouver $f(x) = f(y)$.

Par exemple, si f est une fonction paire, on peut avoir la règle de congruence $x = -y \implies f(x) = f(y)$, ce qui permet au simplificateur de démontrer $f(-3) = f(3)$

Si le simplificateur arrive à un terme sur lequel il n'y a pas de règles applicables (ou les seules règles potentiellement applicables ont une condition dont la preuve a échoué), alors il s'arrête. Si de plus il s'arrête sur un terme de la forme $t = t$, alors il considère son but comme démontré.

Pour la certification d'équivalence de circuits, il est judicieux d'utiliser `simp`, car notre forme normale est calculée à l'aide de règles de réécriture. Cependant, les règles de réécriture indiquées ci-dessus ne sont pas suffisantes pour le bon fonctionnement de `simp`. En effet, il lui faut aussi :

- des règles pour évaluer certaines fonctions (`dom`, `cod`, ...)
- des règles pour faire de l'arithmétique sur les entiers naturels
- des règles pour traiter des inégalités sur les entiers naturels
- des règles pour évaluer des opérations booléennes

Il est aussi nécessaire d'écrire les règles d'une manière appropriée pour `simp` : une règle comme $f(n+1) = g(n)$ ne s'applique pas sur $f(13)$, mais sur $f(12+1)$. Il convient de prendre à la place une règle comme $n \geq 1 \implies f(n) = g(n-1)$

Il est également important de prendre en compte la difficulté posée par la soustraction sur les entiers naturels. En effet, si $n \geq m$, $m - n$ est défini comme étant 0. Par exemple, $1 - 2 = 0$ peut être démontré en Isabelle. Il faut donc exprimer $n - m$ par $\text{nat } (\text{int } n - \text{int } m)$ (int est l'injection des naturels dans les entiers relatifs)

Un autre obstacle rencontré est le fait que on a besoin de deux règles de congruence :

$$\text{NF}(u_1) = \text{NF}(v_1) \implies \text{NF}(u_2) = \text{NF}(v_2) \implies \text{NF}(u_1 \otimes u_2) = \text{NF}(v_1 \otimes v_2) \quad (\text{par_cg})$$

$$\text{NF}(u_1) = \text{NF}(v_1) \implies \text{NF}(u_2) = \text{NF}(v_2) \implies \text{NF}(u_1 \circ u_2) = \text{NF}(v_1 \circ v_2) \quad (\text{seq_cg})$$

Cependant, Isabelle refuse d'appliquer les deux règles dans le même appel à `simp` (je pense que c'est parce que le simplificateur ne regarde que la tête du terme, et ne prend qu'une seule règle de congruence par tête, or les deux termes commencent par `NF`). En effet, un appel à `(simp [...] cong: par_cg seq_cg)` n'applique que la règle `par_cg`, et un appel à `(simp [...] cong: seq_cg par_cg)` n'applique que la règle `seq_cg`.

J'ai contourné ce problème en utilisant des `congrproc` (ou procédures de congruence). Il est alors possible de donner un pattern plus sophistiqué pour chaque règle.

Par exemple, pour la règle `seq_cg`, j'ai écrit la règle

```
simproc_setup passive weak_congrproc cong_seq (<NF (Seq x y) >) =
  <(K (K (K (SOME @{thm seq_cg [cong_format]})))) >
```

Dans le code ci-dessus, une `simproc` (procédure de simplification) nommée `cong_seq` est définie. Le mot clé **weak_congrproc** sert à déclarer que cette procédure est en réalité une procédure de congruence, et la partie `weak` signale à Isabelle que cette procédure ne s'occupe pas de simplifier l'intérieur du terme.

La partie `<NF (Seq x y) >` indique à Isabelle que cette procédure doit être invoquée lorsque le simplificateur essaie de prouver `NF (Seq x1 x2) = NF (Seq y1 y2)` (où la fonction `Seq u v` représente $u \circ v$). Cela permet notamment de la distinguer de l'autre règle qui doit être appliquée sur `<NF (Par x y) >`

L'expression `(K (K (K (SOME @{thm seq_cg [cong_format]}))))` est l'équivalent d'une fonction OCaml `fun _ -> fun _ -> fun _ -> Some seq_cg` qui renvoie le théorème correspondant à la règle de congruence voulue. Il est possible de renvoyer un théorème qui dépend des trois arguments, mais ce n'est ici pas nécessaire.

2.2 Equivalence des PRO

Pour la simplification des **PRO**_Σ, on utilise l'ensemble de règles de simplification suivant :

```
lemmas PRO_simps =
  R1_NF R2_NF R3_NF R4_NF R5_NF R6_NF R7_NF R8_NF R9_NF R10_NF R11_NF

  admissible.intros                (* rules for admissibility *)
  dom.simps cod.simps def.simps   (* rules for dom, cod & def *)
  trm.disc(5-8)                   (* rules for is_Id *)

  arith_simps                     (* rules for arithmetic *)
  of_nat_0 of_nat_1 of_nat_numeral (* rules for int *)
  nat_0 nat_numeral               (* rules for nat *)
  num1                            (* rule to handle 1 *)
```

```

rel_simps(1-71)          (* rules for integer inequalities *)
simp_thms                (* rules for boolean calculus *)

```

où les règles $R1_NF$ jusqu'à $R11_NF$ sont les règles de réduction définies en [Figure 2](#), et les autres règles permettent l'évaluation de fonction et le calcul sur les entiers et les booléens

J'ai également écrit une preuve pour certifier l'équivalence entre deux termes t_1 et t_2 :

```

lemma <NF t1 = NF t2>
proof -
  have pre_eq: <NF (preproc t1) = NF (preproc t2)>
    apply (simp only: preproc.simps)
    including no_limit by (simp only: PRO_simps [[simproc cong_par
      cong_seq]])?
  have t1_pre: <NF (preproc t1) = NF t1> using pre by blast
  have t2_pre: <NF (preproc t2) = NF t2> using pre by blast
  show ?thesis
    using t1_pre t2_pre pre_eq by argo

```

qed

Plusieurs détails sont ici importants :

- La fonction `preproc` rajoute quelques circuits vides (id_0) pour éviter quelques cas problématiques
- **including** `no_limit` permet à la tactique `simp` d'aller à profondeur très grande dans le résultat (pour que `simp` n'abandonne pas sa simplification)
- Le point d'interrogation présent à la fin de la ligne 5 permet de gérer correctement le cas où les termes t_1 et t_2 sont exactement égaux. Dans ce cas, le but est déjà résolu à la ligne 4, et la ligne 5 n'est plus nécessaire.

Par ailleurs, j'ai défini `no_limit` comme :

```

bundle no_limit = [[simp_depth_limit=10000]]

```

Le nombre 10000 doit être choisi assez grand pour pouvoir être sûr de que `simp` n'abandonne pas.

2.3 Equivalence des permutations

De même, pour les `Perm`, on utilise cet ensemble de règles :

```

lemmas PRO_Perm_simps =
  R1_to_perm  R2_to_perm  R3_to_perm  R4_to_perm  R5_to_perm
  R6_to_perm  R7_to_perm  R8_to_perm  R9_to_perm  R12_to_perm
  R13_to_perm R14_to_perm R15_to_perm_simp R16_to_perm_simp
  R17_to_perm_simp R18_to_perm_simp R19_to_perm_simp
  R20_to_perm_simp R21_to_perm_simp R22_to_perm_simp

  sadmissible.intros          (* rules for admissibility *)
  dom.simps cod.simps def.simps (* rules for dom, cod & def *)
  trm.disc                   (* rules for is_Id *)

  arith_simps                (* rules for arithmetic *)
  of_nat_0 of_nat_1 of_nat_numeral (* rules for int *)

```

```

nat_0 nat_numeral          (* rules for nat *)
num1                       (* rule to handle 1 *)

rel_simps                  (* rules for integer inequalities *)
simp_thms                  (* rules for boolean calculus *)

```

où les règles `R1_to_perm` jusqu'à `R22_to_perm_simp` sont les règles de réduction du système défini en [Figure 2](#) et [Figure 3](#), et les autres règles permettent l'évaluation de fonction et le calcul sur les entiers et les booléens.

Et pour certifier l'équivalence entre deux termes t_1 et t_2 :

```

lemma <CF [[t1]] = CF [[t2]] >
proof -
  have pre_eq: <[[preproc t1]] = [[preproc t2]] >
    apply (simp only: preproc.simps)
    including no_limit by (simp only: PRO_Perm_simps [[simproc
      cong_perm_par cong_perm_seq]])?
  have <sadmissible t1> including no_limit by (simp only:
    PRO_Perm_simps)
  also have <sadmissible t2> including no_limit by (simp only:
    PRO_Perm_simps)
  ultimately have <[[t1]] = [[t2]] >
    using pre_eq perm_preproc[of t1] perm_preproc[of t2] by argo
  then show ?thesis by argo

qed

```

Ici, le prédicat `sadmissible` représente le fait qu'un circuit est bien formé, i.e. que $\text{cod}(u) = \text{dom}(v)$ lorsqu'on a $u \circ v$

3 Tests de performance

3.1 Méthodologie

Pour tester la rapidité de certification de circuits, des circuits ont été générés. Pour cela un script qui génère des paires de circuits équivalents (merci à Julie Cailler pour celui-ci) a été utilisé. Les paires ainsi générées sont ensuite classées par taille et regroupées en lots de 50 en 50 (c'est-à-dire un lot pour les tailles de 0 à 49, un pour les tailles de 50 à 99, etc...), où la taille est définie par le nombre de $\otimes$ et de $\circ$ dans les 2 circuits.

Ensuite, sont générés pour chaque lot de taille un ensemble de 50 exemples en les tirant au hasard (avec remplacement).

Isabelle est ensuite lancée sur les 50 exemples en même temps (ce qui permet à Isabelle de potentiellement paralléliser ses calculs) et le temps final d'exécution est récupéré.

Le script permettant de générer les circuits est un script aléatoire et ne permet pas de contrôler la taille de sortie des circuits. De plus, pour les très grands circuits, le script peut prendre beaucoup de temps. Pour cette raison, le nombre d'exemples générés dépend de la taille.

Un des problèmes de cette méthode est que la précision de la mesure dépend beaucoup du nombre d'exemples générés : s'il y en a plus de 200, la répartition est proche de l'uniforme, mais lorsqu'il y en a moins que 10, des biais peuvent exister. Il faut donc

s'attendre à ce que les tests soient moins précis lorsque les circuits sont plus grands (les exemples étant plus longs à générer, on en possède moins)

Un autre problème qui perturbe la mesure est que Isabelle a besoin de s'initialiser et de lire les fichiers qui établissent la théorie permettant de faire la preuve. Ceci prend un temps constant (car ces fichiers ne dépendent pas du circuit testé) qui devrait être assez peu significatif, puisqu'il n'y a qu'une seule étape d'initialisation pour les 50 exemples.

Il est également important de noter qu'Isabelle a pu tourner en parallèle sur 8 threads, ce qui lui permet de traiter plusieurs circuits en même temps.

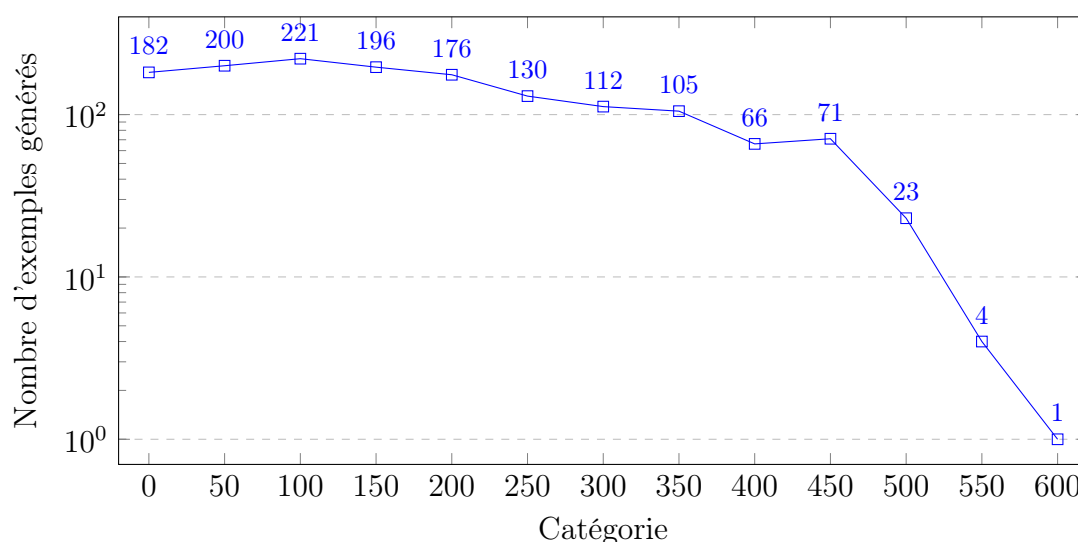
Tous les exemples ont été lancés sur Grid5000 ([lien](#)), effectués sans qu'il y ait d'autres processus en même temps, et tous les tests appartenant à une même famille ont été réalisés durant la même session.

3.2 Résultats pour les PRO

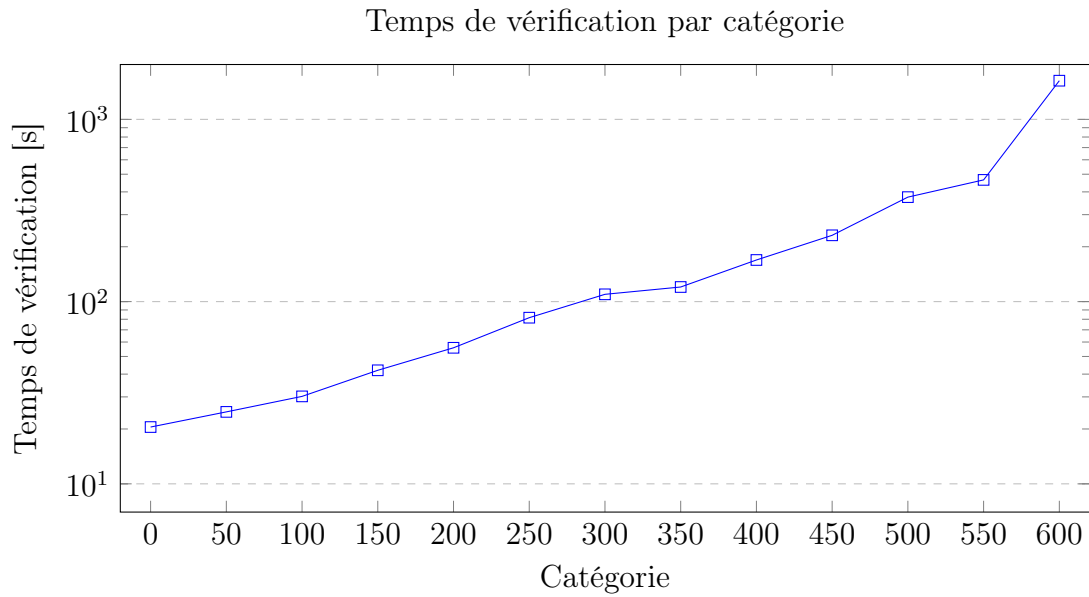
Le nombre d'exemples générés chute très rapidement à partir de la taille 500, du fait du temps de génération.

Le graphe ci-dessous montre le nombre d'exemples générés en fonction de la catégorie. Pour aider à la lisibilité, l'axe des ordonnées est ici logarithmique.

Nombre d'exemples générés par catégorie



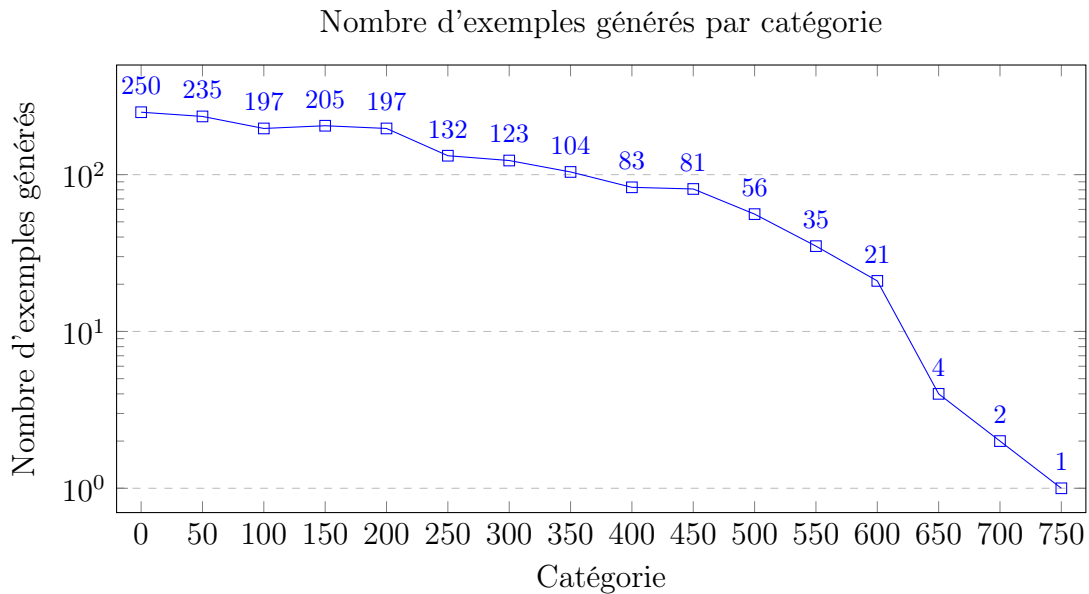
Le graphe ci-dessous montre temps de calcul, également avec l'ordonnée logarithmique. Il est important de noter que la mesure pour la catégorie 600 est probablement imprécise, car on ne dispose que d'un seul exemple (le test s'est donc déroulé avec 50 copies du même exemple, ce qui peut ne pas être représentatif d'un échantillon uniforme).



3.3 Résultats pour les permutations

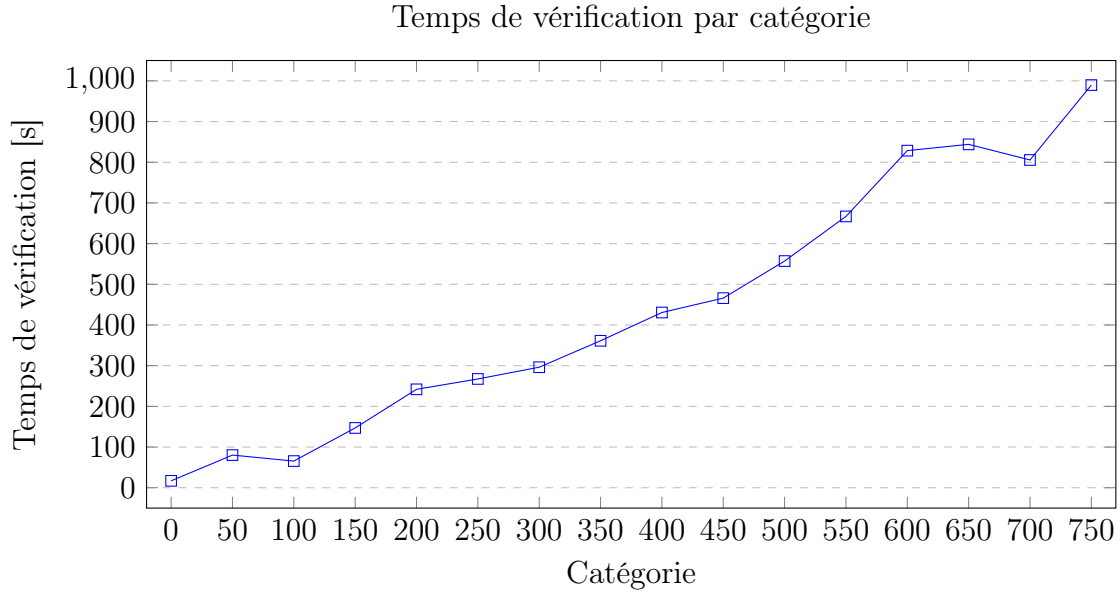
Ici le nombre d'exemples chute pour les catégories supérieures à 650. Ce nombre est plus grand que pour les PRO car il semble qu'il soit plus simple de générer des circuits de type permutation que des circuits de type PRO.

Comme le graphe précédent, les ordonnées sont ici logarithmiques.



Le temps de calcul pour les permutations semble ne pas croître aussi vite que celui pour les PRO. En conséquence, le graphe ci dessous est donné avec l'axe des ordonnées linéaire et non logarithmique.

Il est important de noter que les points pour les catégories 650, 700 et 750 ne sont probablement pas représentatifs d'un échantillon uniforme, vu leur faible nombre d'exemples.



4 Extension aux PROP

Afin d'espérer pouvoir obtenir un résultat similaire sur les $\mathbf{PROP}_{\Sigma, \emptyset}$, on veut définir une forme normale.

Remarque 4.0.1. *Comme les $\mathbf{PROP}_{\Sigma}$ sont des $\mathbf{PRO}_{\Sigma \cup \Sigma_0}$ (avec des générateurs supplémentaires), on peut tout à fait appliquer la fonction de normalisation des $\mathbf{PRO}_{\Sigma \cup \Sigma_0}$ sur les $\mathbf{PROP}_{\Sigma}$, ce qui donne un circuit séparé en couches.*

Cela revient, en outre, à considérer que les swap sont des générateurs quelconques (des boîtes noires, en quelque sorte) et à effectuer le processus de normalisation des $\mathbf{PRO}_{\Sigma \cup \Sigma_0}$ dessus.

On considèrera donc pendant toute cette partie que les circuits sur lesquels on travaille sont déjà séparés en couches.

4.1 Numérotation

Pour aider à définir une forme normale, on va ajouter des informations à notre circuit. Cela permet de donner des informations globales sur le circuit de manière locale.

Remarque 4.1.1. *On peut voir un circuit comme un graphe, dont les noeuds sont les portes et les arêtes sont les fils (orientées de gauche à droite), avec un ordre sur les arêtes.*

Etant donné un circuit t , on note son graphe correspondant $G(t)$

Théorème 4.1.2. $t_1 \simeq t_2 \iff G(t_1) = G(t_2)$

Démonstration. C'est le théorème de cohérence (théorème 3.12) de [5] □

Corollaire 4.1.3. *Toute quantité qui ne dépend que de $G(t)$ est invariante pour toute la classe d'équivalence de t*

Définition 4.1.4. *On définit la profondeur $\rho(u)$ d'une porte $u \in \Sigma$ dans un circuit comme étant le nombre maximal de portes traversées vers la gauche.*

Autrement dit, si les portes précédant v sont $u_1, \dots, u_n$, alors

$$\rho(v) = \begin{cases} 0 & \text{si } n = 0 \\ 1 + \max_{1 \leq i \leq n} \rho(u_i) & \text{sinon} \end{cases}$$

Exemple 4.1.5. Voir la [Section 7.1](#)

Lemme 4.1.6. Comme la notion de "précéder" ne dépend que du graphe sous-jacent, la profondeur ainsi définie est indépendante de l'élément de la classe d'équivalence choisie.

Définition 4.1.7. On note la "vraie profondeur" $p(v)$ d'un générateur (c'est-à-dire une porte ou bien un swap) par l'équation suivante :

$$p(v) = \begin{cases} 2 \times \rho(v) + 1 & \text{si } v \text{ est une porte} \\ 2 \times \max_{1 \leq i \leq n} \rho(u_i) & \text{si } v \text{ est un swap et } (u_i)_i \text{ les portes le précédant} \end{cases}$$

Exemple 4.1.8. Voir la [Section 7.2](#)

Remarque 4.1.9. Avec cette notion de profondeur, les swaps ont toujours une profondeur paire tandis que les portes ont toujours une profondeur impaire. Ainsi, un swap et une porte n'ont jamais la même profondeur.

Définition 4.1.10. On définit la priorité d'un fil en faisant un parcours en profondeur sur le graphe. On obtient ainsi un ordre topologique sur les fils.

Exemple 4.1.11. Voir la [Section 7.3](#)

Lemme 4.1.12. Comme cet ordre ne dépend que du graphe sous-jacent, il est équivalent pour tout élément de la classe d'équivalence.

Lemme 4.1.13. On peut donc munir un circuit des profondeurs et des priorités sans toutefois modifier les classes d'équivalence.

4.2 Forme normale

Définition 4.2.1. On dit d'un circuit (annoté) est en forme normale si et seulement si :

- la profondeur de ses couches successives est croissante
- pour deux couches de même profondeur qui se suivent et contenant des portes, la porte dont les fils de sortie ont la priorité la plus faible se trouve avant.
- pour une profondeur paire donnée (cela ne concerne donc que les swaps), l'ensemble des couches de cette profondeur représente une permutation qui est en forme normale selon la fonction de normalisation des permutations.

Exemple 4.2.2. Voir la [Section 7.4](#)

Lemme 4.2.3. La forme normale existe, et est unique.

i.e. si t est un circuit, il existe un unique circuit $t' \simeq t$ tel que t' est en forme normale.

Démonstration. Existence : On peut construire la forme normale à l'aide du graphe $G(t)$.

En effet, il suffit d'itérer sur chaque profondeur, et de trier les priorité des portes qui s'y trouvent. Cela force les permutations entre ces portes, qui peuvent être construites en

prenant un circuit qui représente cette permutation et en lui appliquant la forme normale des permutations.

Unicité : Il n'y a qu'une seule manière d'arranger les portes pour chaque profondeur (selon les priorités).

De plus, à une profondeur donnée, les swap n'ont qu'un seul arrangement possible (par l'unicité de la forme normale des permutations)

La forme normale ainsi définie est donc unique □

Définition 4.2.4. On note NF la procédure qui transforme t en forme normale.

Propriété 4.2.5. $NF(NF(t)) = NF(t)$

Démonstration. Par définition de NF , on a $\forall u, NF(u) \simeq u$

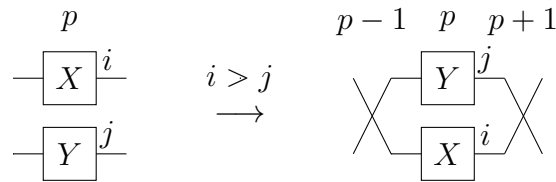
Par unicité de la forme normale, comme $NF(t)$ est en forme normale, on a bien $NF(NF(t)) = NF(t)$ □

Lemme 4.2.6. $t_1 \simeq t_2 \iff NF(t_1) = NF(t_2)$

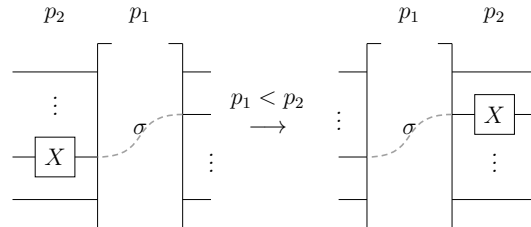
Démonstration. Par unicité de la forme normale. □

4.3 Système de réécriture

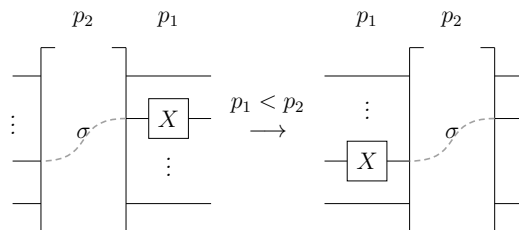
Définition 4.3.1. On pose le système de réécriture suivant sur les circuits numérotés



où i, j sont les priorités des fils sortants et les portes X et Y sont à la profondeur p



où σ est une permutation et p_1, p_2 des profondeurs



où σ est une permutation et p_1, p_2 des profondeurs

Ce système est très intuitif dans le sens qu'il n'est pas assez formel pour être utilisé par la machine, mais il est toutefois possible de raisonner sur ce système graphiquement. En particulier, ce système demande sans l'expliciter de faire "coulisser" les portes de plus faible priorité vers la gauche. Une version plus formelle de ce système est décrite ci-dessous, même si elle perd le côté "intuition graphique"

Lemme 4.3.2. *Tout circuit qui n'est pas en forme normale possède au moins une règle applicable*

Démonstration. Supposons qu'aucune règle ne soit applicable.

Comme les règles 2 et 3 ne sont pas applicables, les portes sont triées par profondeur croissante.

Les portes adjacentes sont alors de même profondeur, i.e. la règle 1 s'applique si et seulement si il existe 2 portes adjacentes de priorité décroissante (de haut en bas)

Or la règle 1 ne s'applique pas, d'où les portes sont triées par profondeur et par priorité. Le circuit est donc en forme normale. $\square$

Lemme 4.3.3. *Le système est terminant*

Démonstration. Soit u un circuit

Soit n le nombre d'applications minimal des règles 2 et 3 pour obtenir à partir de u un circuit où toutes les portes sont ordonnées par profondeur

Soit m le nombre d'applications minimal de la règle 1 pour obtenir à partir de u un circuit où toutes les priorités sont ordonnées.

On pose alors la mesure $\mu(u) = (n, m)$, avec l'ordre lexicographique.

On remarque que l'application de la règle 2 ou de la règle 3 fait décroître n , et que l'application de la règle 1 ne change pas n et fait décroître m .

Comme l'ordre lexicographique sur $\mathbb{N} \times \mathbb{N}$ est un bon ordre, il n'existe pas de suite infinie strictement décroissante de paires d'entiers.

Le système de réécriture est donc terminant. $\square$

Lemme 4.3.4. *Le système est confluent*

Démonstration. Soit t un circuit et supposons que l'on a $t_1 \leftarrow t \rightarrow t_2$.

Comme le système est terminant, il existe t'_1 tel que $t_1 \xrightarrow{*} t'_1$ et aucune règle ne s'applique à t'_1 .

D'après le lemme ci-dessus, t'_1 est donc en forme normale, i.e. $t'_1 = \text{NF}(t_1)$.

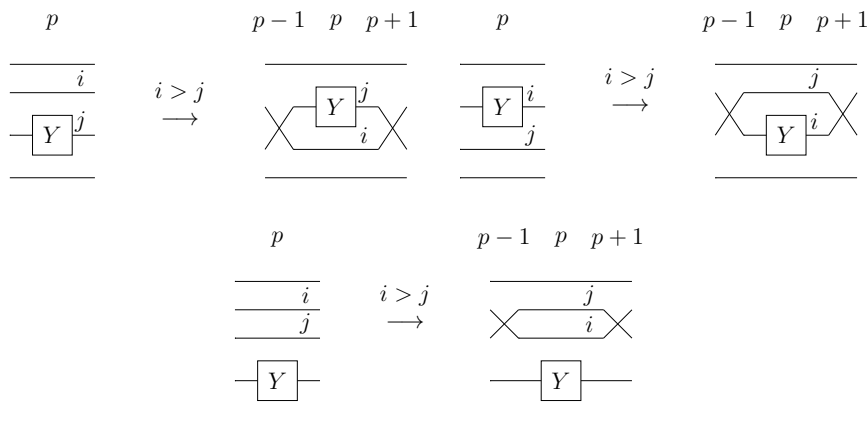
On a donc $t_1 \xrightarrow{*} \text{NF}(t_1)$

De même, on a $t_2 \xrightarrow{*} \text{NF}(t_2)$

Or $t_1 \simeq t \simeq t_2$, d'où $\text{NF}(t_1) = \text{NF}(t_2)$

On a donc bien la confluence du système de réécriture. $\square$

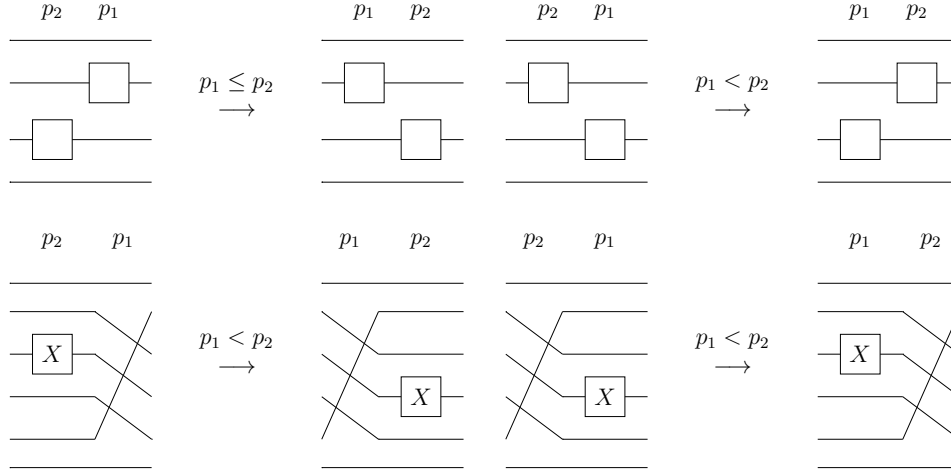
Remarque 4.3.5. *La règle 1 peut être remplacée par les 3 règles*



où Y est une porte, i, j sont les priorités des fils sortants et p est la profondeur de la couche considérée.

Note : Pour la troisième règle, la position de la porte Y par rapport aux fils considérés n'est pas importante, donc cette règle s'applique aussi si la porte Y se trouve au dessus des fils de priorité i et j

Remarque 4.3.6. *Les règles 2 et 3 peuvent être remplacées par les 4 règles*



où les boîtes vides représentent des swaps ou des portes, et les boîtes nommés X représentent des portes uniquement. De plus, p_1 et p_2 représentent ici les profondeurs des couches considérées.

Définition 4.3.7. *On pose le système de réécriture (formel) sur les circuits numérotés composé des 7 règles décrites ci-dessus, ainsi que des règles R14 à R22 des permutations (voir Figure 3)*

Remarque 4.3.8. *Je n'ai pas démontré la terminaison du système formel, mais la terminaison du système intuitif donne une bonne indication que le système que j'ai ici décrit est effectivement terminant*

5 Conclusion

Il reste sur le sujet plusieurs choses à faire, comme formaliser le système de réécriture en Isabelle, prouver sa terminaison et sa confluence, et enfin implémenter le certificat permettant de décider l'équivalence des $\mathbf{PROP}_\Sigma$.

Une possible extension de ce système pourrait également accommoder les termes de la forme $\sigma_{n,m}$, où n et m sont des variables et non des entiers fixés.

6 Remerciements

Je voudrais remercier mon encadrante de stage (Sophie Turret), les autres personnes travaillant sur le sujet pour leur indications et appréciations (Julie Cailler et Noé Delorme), toute l'équipe VERIDIS m'ayant accueilli au laboratoire, et le laboratoire LORIA.

Je remercie aussi Grid5000 pour m'avoir permis de faire tourner mes tests de performance.

Références

- [1] Bonchi, F., Gadducci, F., Kissinger, A., Sobocinski, P., Zanasi, F. : String diagram rewrite theory II : Rewriting with symmetric monoidal structure. *Mathematical Structures in Computer Science* **32**(4), 511–541 (April 2022), <http://dx.doi.org/10.1017/S0960129522000317>
- [2] Cailler, J., Delorme, N., Perdrix, S., Tournet, S. : Towards Term-based Verification of Diagrammatic Equivalence (2026), <https://arxiv.org/abs/2602.11035>
- [3] Clément, A., Heurtel, N., Mansfield, S., Perdrix, S., Valiron, B. : A complete equational theory for quantum circuits. In : *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)* (2023). <https://doi.org/10.1109/LICS56636.2023.10175801>, <https://arxiv.org/abs/2206.10577>
- [4] Mondada, L., Andrés-Martínez, P. : Scalable Pattern Matching in Computation Graphs. *Electronic Proceedings in Theoretical Computer Science* **417**, 71–95 (March 2025), <http://dx.doi.org/10.4204/eptcs.417.5>
- [5] Selinger, P. : A Survey of Graphical Languages for Monoidal Categories, pp. 289–355. Springer Berlin Heidelberg (2010), http://dx.doi.org/10.1007/978-3-642-12821-9_4

7 Annexes

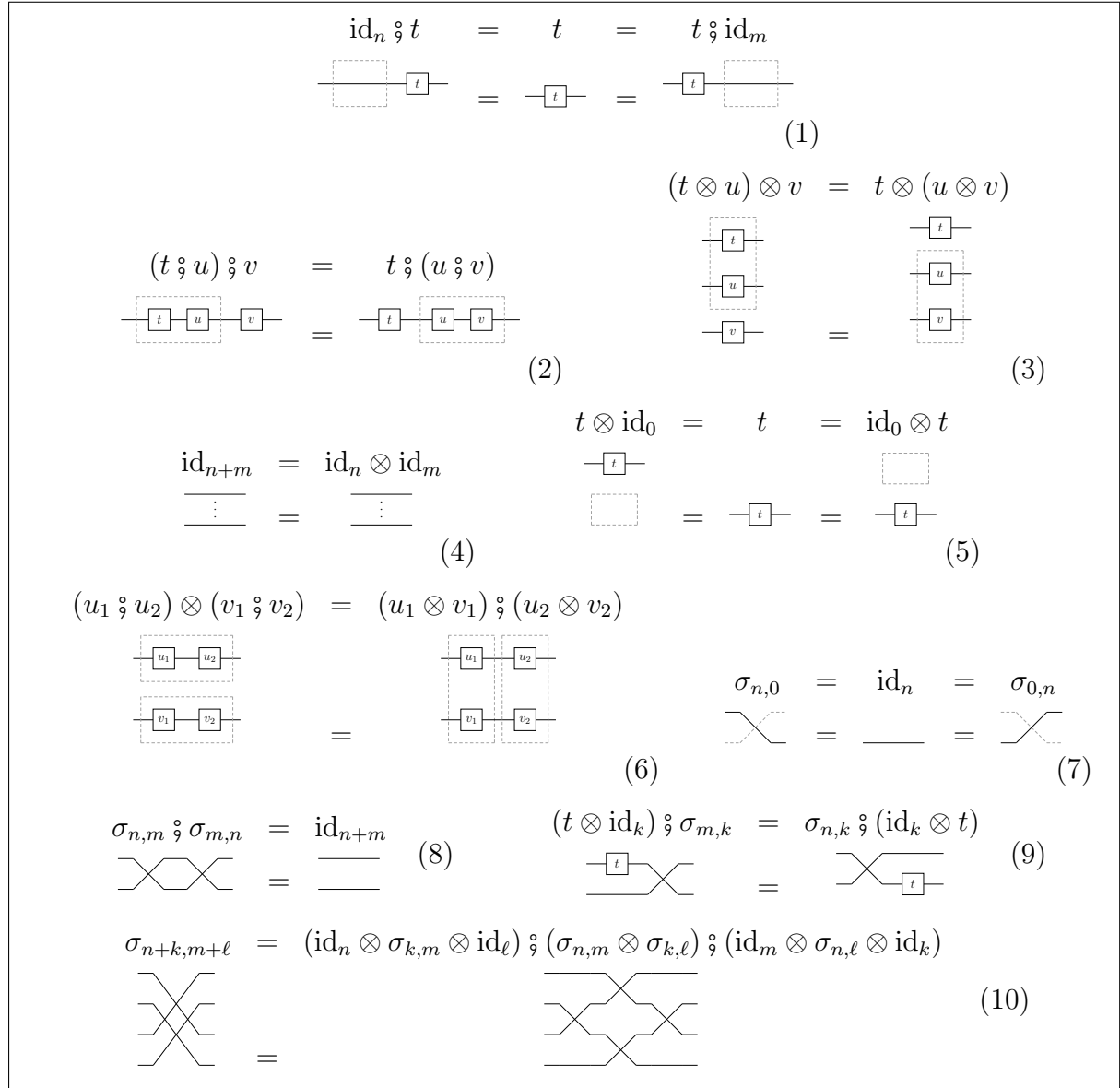


FIGURE 1 – Egalités des PRO et PROP

$$(t \circ u) \circ v \longrightarrow t \circ (u \circ v) \quad (\text{R1}) \qquad (t \otimes u) \otimes v \longrightarrow t \otimes (u \otimes v) \quad (\text{R2})$$

$$\text{id}_n \circ t \longrightarrow t \quad (\text{R3}) \qquad t \circ \text{id}_n \longrightarrow t \quad (\text{R4})$$

$$\text{id}_n \otimes \text{id}_m \longrightarrow \text{id}_{n+m} \quad (\text{R5}) \qquad \text{id}_n \otimes (\text{id}_m \otimes t) \longrightarrow \text{id}_{n+m} \otimes t \quad (\text{R6})$$

$$u \otimes v \longrightarrow (u \otimes \text{id}_{\text{dom}(v)}) \circ (\text{id}_{\text{cod}(u)} \otimes v) \text{ if } u \neq \text{id}_n \wedge v \neq \text{id}_m \quad (\text{R7})$$

$$\text{id}_n \otimes (u \circ v) \longrightarrow (\text{id}_n \otimes u) \circ (\text{id}_n \otimes v) \quad (\text{R8})$$

$$(u \circ v) \otimes \text{id}_n \longrightarrow (u \otimes \text{id}_n) \circ (v \otimes \text{id}_n) \quad (\text{R9})$$

$$L_{\ell_1}^{k_1}(g_1) \circ L_{\ell_2}^{k_2}(g_2) \longrightarrow L_{\ell_2 + \text{def}(g_1)}^{k_2}(g_2) \circ L_{\ell_1}^{k_1 - \text{def}(g_2)}(g_1) \quad (\text{R10})$$

if $k_1 \geq k_2 + \text{dom}(g_2)$

$$L_{\ell_1}^{k_1}(g_1) \circ (L_{\ell_2}^{k_2}(g_2) \circ t) \longrightarrow L_{\ell_2 + \text{def}(g_1)}^{k_2}(g_2) \circ (L_{\ell_1}^{k_1 - \text{def}(g_2)}(g_1) \circ t) \quad (\text{R11})$$

if $k_1 \geq k_2 + \text{dom}(g_2)$

FIGURE 2 – Système de réécriture pour les PRO

$$\sigma_{0,n} \longrightarrow \text{id}_n \quad (\text{R12}) \qquad \sigma_{n,0} \longrightarrow \text{id}_n \quad (\text{R13})$$

$$\sigma_{n+1,m+2} \longrightarrow (\sigma_{n+1,m+1} \otimes \text{id}_1) \circ (\text{id}_{m+1} \otimes \tau_{n+1}) \quad (\text{R14})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ L_{\ell_2}^{k_2}(\tau_{d_2}) &\longrightarrow L_{\ell_2}^{k_2}(\tau_{d_2}) \circ L_{\ell_1-1}^{k_1+1}(\tau_{d_1}) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_2 \leq k_1 < k_2 + d_2 - d_1 \end{aligned} \quad (\text{R15})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ (L_{\ell_2}^{k_2}(\tau_{d_2}) \circ t) &\longrightarrow L_{\ell_2}^{k_2}(\tau_{d_2}) \circ (L_{\ell_1-1}^{k_1+1}(\tau_{d_1}) \circ t) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_2 \leq k_1 < k_2 + d_2 - d_1 \end{aligned} \quad (\text{R16})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ L_{\ell_2}^{k_2}(\tau_{d_2}) &\longrightarrow L_{\ell_2+1}^{k_2}(\tau_{d_2-1}) \circ L_{\ell_1}^{k_1+1}(\tau_{d_1-1}) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_2 \leq k_1 \wedge k_2 + d_2 - d_1 \leq k_1 < k_2 + d_2 \end{aligned} \quad (\text{R17})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ (L_{\ell_2}^{k_2}(\tau_{d_2}) \circ t) &\longrightarrow L_{\ell_2+1}^{k_2}(\tau_{d_2-1}) \circ (L_{\ell_1}^{k_1+1}(\tau_{d_1-1}) \circ t) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_2 \leq k_1 \wedge k_2 + d_2 - d_1 \leq k_1 < k_2 + d_2 \end{aligned} \quad (\text{R18})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ L_{\ell_2}^{k_2}(\tau_{d_2}) &\longrightarrow L_{\ell_1}^{k_2}(\tau_{d_1+d_2}) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_1 = k_2 + d_2 \end{aligned} \quad (\text{R19})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ (L_{\ell_2}^{k_2}(\tau_{d_2}) \circ t) &\longrightarrow L_{\ell_1}^{k_2}(\tau_{d_1+d_2}) \circ t \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_1 = k_2 + d_2 \end{aligned} \quad (\text{R20})$$

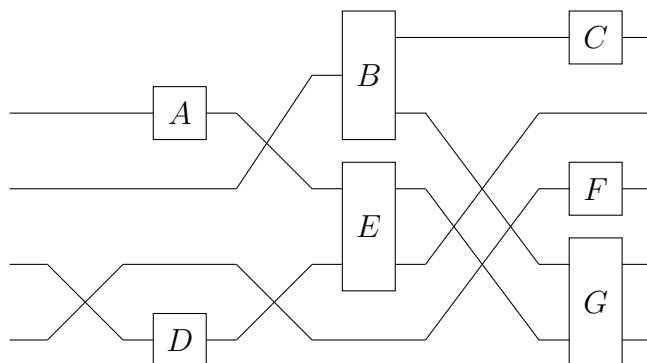
$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ L_{\ell_2}^{k_2}(\tau_{d_2}) &\longrightarrow L_{\ell_2}^{k_2}(\tau_{d_2}) \circ L_{\ell_1}^{k_1}(\tau_{d_1}) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_1 > k_2 + d_2 \end{aligned} \quad (\text{R21})$$

$$\begin{aligned} L_{\ell_1}^{k_1}(\tau_{d_1}) \circ (L_{\ell_2}^{k_2}(\tau_{d_2}) \circ t) &\longrightarrow L_{\ell_1}^{k_1}(\tau_{d_1}) \circ (L_{\ell_2}^{k_2}(\tau_{d_2}) \circ t) \\ \text{if } d_1 \geq 1, d_2 \geq 1 \text{ and } k_1 > k_2 + d_2 \end{aligned} \quad (\text{R22})$$

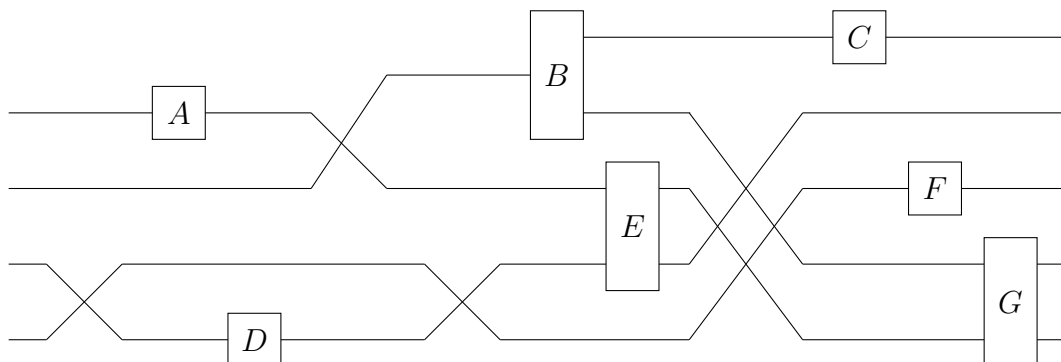
FIGURE 3 – Système de réécriture pour les permutations

7.1 Exemple : profondeur des portes

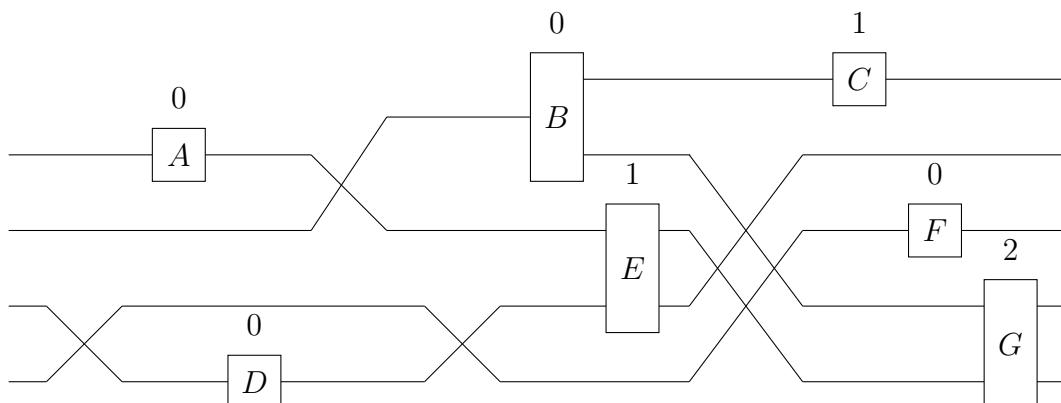
On utilisera pour illustrer la notion de profondeur ce circuit :



Pour plus de lisibilité, on l'éclate en couches (c'est-à-dire un seul élément par colonne)



On peut alors annoter les portes à l'aide de leur profondeur :

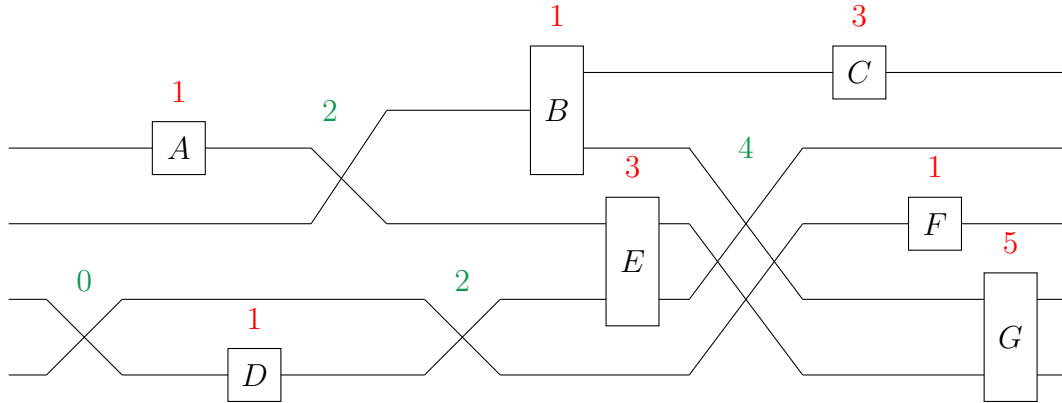


Il faut en particulier noter que la profondeur désigne le nombre maximal de portes à traverser depuis la gauche.

Par exemple, la porte G est bien de profondeur 2, car elle peut être atteinte en traversant la porte A , puis la porte E .

7.2 Exemple : vraie profondeur

En reprenant le circuit précédant, on peut donner les vraies profondeurs :

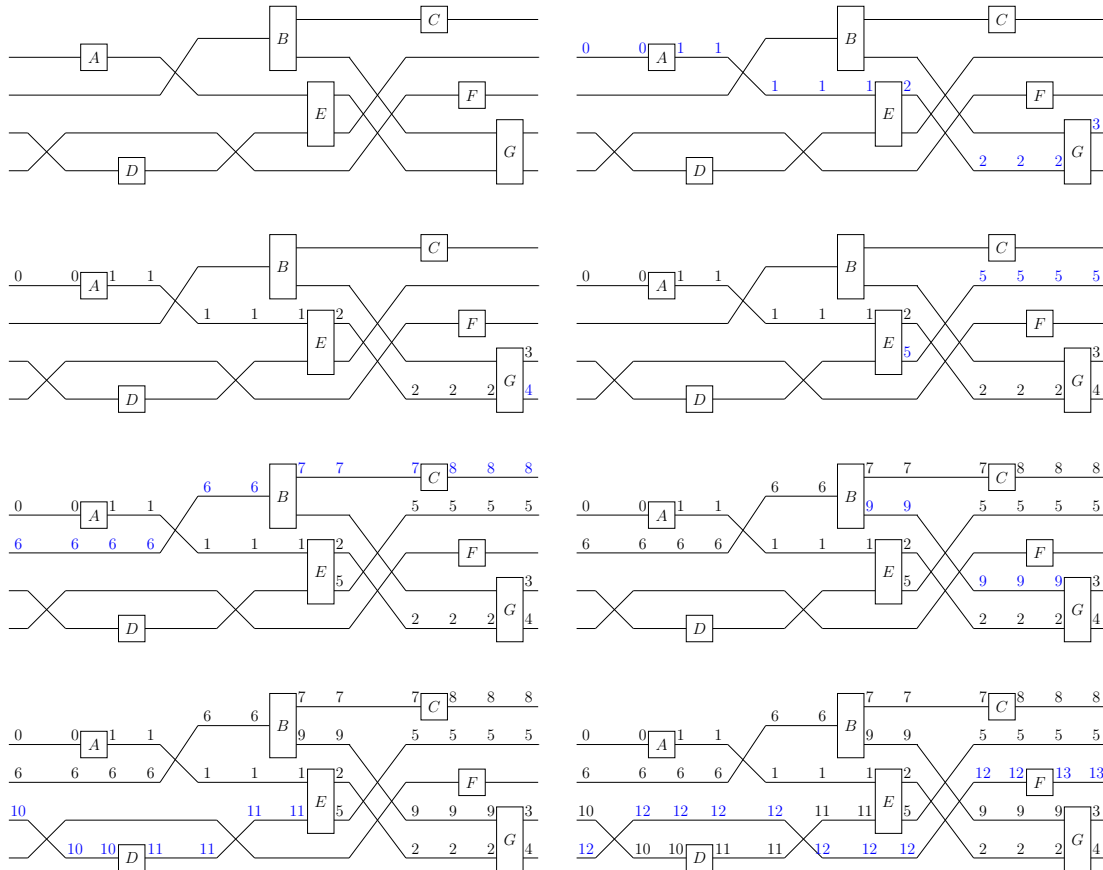


Il faut noter que la profondeur des swaps n'influence pas celle des portes.

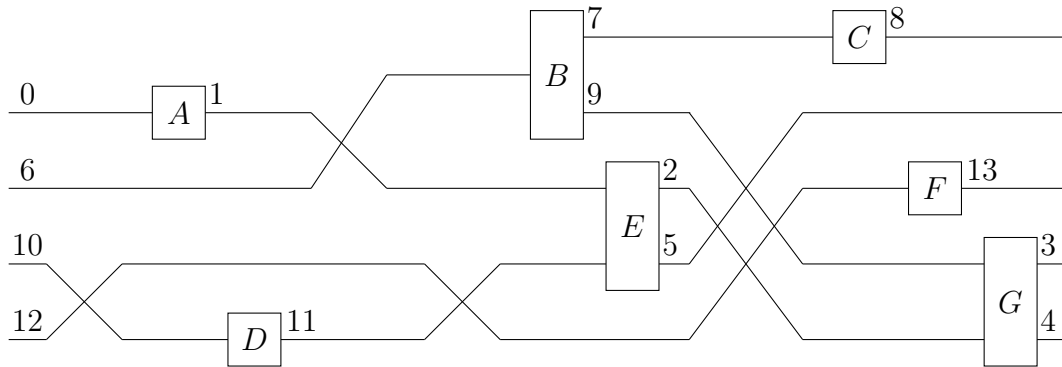
Par exemple, la porte F n'est que de profondeur 1, alors qu'elle suit un swap de profondeur 4.

7.3 Exemple : priorité des fils

Toujours avec le même exemple, voici le déroulé du parcours en profondeur sur le circuit, ainsi que l'assignation des priorités aux fils (tableau à lire de gauche à droite, ligne par ligne, les différences à chaque étape sont mises en bleu)

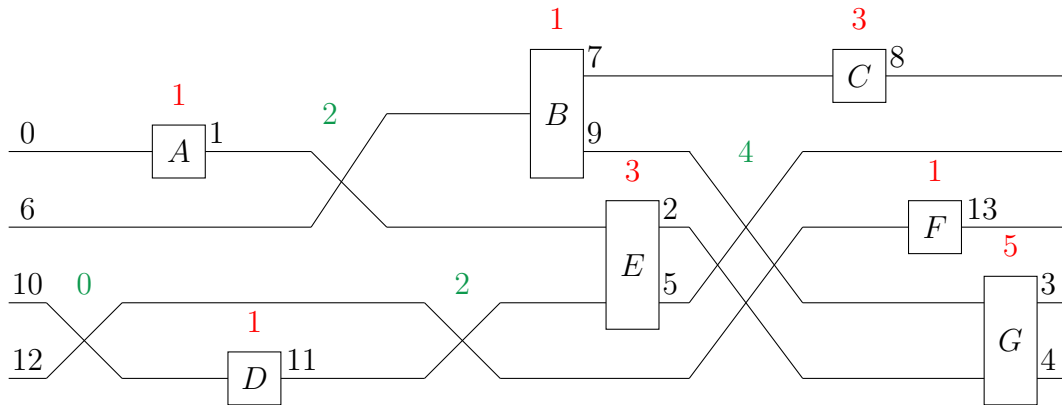


Pour éviter de surcharger le circuit, on ne garde pour chaque valeur que les annotations les plus à gauche, laissant les autres implicites.

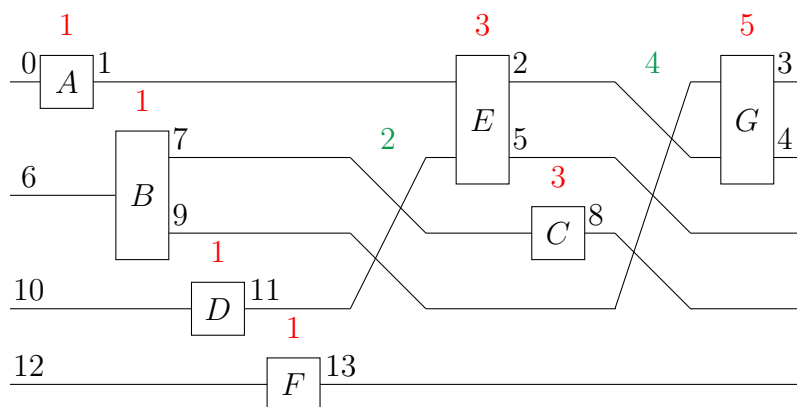


7.4 Exemple : forme normale

En combinant les annotations de profondeur et de priorité sur l'exemple précédent, on obtient :



La forme normale correspondant à ce circuit est donc :



On remarque que les annotations de profondeur sur les portes ainsi que celles de priorité sur les fils restent inchangées.

Par contre, comme la normalisation peut créer, combiner et enlever des swaps, il n'en est pas de même pour la profondeur de ces derniers